Hue_{32}

A Device-Independent Human-Readable Color Notation System

Specification v1.4 — Draft

Abstract

Hue_{32} is a human-readable color notation system designed to make color codes intuitively understandable without specialized training. A color is described by five appearance axes — three color axes (Hue, Shade, Tone) and two material axes (Luster, Fluorescence) — each on a 1–32 scale. The scale of 32 was chosen to match the limits of human color discrimination: research suggests trained observers can distinguish approximately 25–32 lightness steps and a similar number of chroma steps under normal viewing conditions, making every integer step on the scale perceptually meaningful. The notation is device-independent, anchored to the OKLCh perceptual color space, and is interoperable with color systems expressible in CIE XYZ, including RGB, CMYK, Munsell, and spectral data. The first three axes — Hue, Shade, and Tone — capture chromatic appearance and are derivable from any digital color source. The last two axes — Luster and Fluorescence — describe how light interacts with the physical surface and require direct measurement; they cannot be derived from digital formats.

The system is designed to serve as a common reference notation between existing color systems, enabling unambiguous communication of color across industries that currently use incompatible formats. A paint manufacturer using NCS, a printer using CMYK, and a screen designer using sRGB can express the same color in Hue_{32} notation and each convert it to their own system — provided the source color is within the gamut of CIE XYZ, which encompasses all current practical color systems.

Higher precision variants Hue_{64} and Hue_{192} extend the system for professional and industrial use through decimal subdivision of the same scale, maintaining full backward compatibility.

1. Motivation

Existing color systems fall into two categories: device-specific formats (RGB, CMYK, HSL) that are precise but not human-readable, and named systems (Pantone, RAL, Munsell) that are human-readable but require lookup tables and are not mathematically interoperable.

No current system allows a person to read a color code and form an accurate mental image of the color without specialized training. Hue_{32} addresses this gap by encoding color in terms of perceptual properties — which hue family, how light or dark, how vivid or muted — in a compact and learnable notation.

Secondary motivations include:

- Providing a common reference notation that physical product manufacturers, print designers, and screen designers can use to communicate color across systems that are otherwise incompatible.
- Defining a notation whose base scale of 32 steps matches the limits of human color discrimination, so that every integer step is perceptually meaningful, while scaling gracefully to instrument precision through decimal subdivision.
- Establishing an extensible framework where future axes (e.g. surface finish derived from spectral data) can be added without breaking existing notation.

---

2. Appearance Axes

Every Hue_{32} color is described by five appearance axes, divided into two groups. Color axes — Hue, Shade, and Tone — describe chromatic appearance and are derivable from any digital color source. Material axes — Luster and Fluorescence — describe how light interacts with the physical surface and require direct measurement. All axes use a 1–32 integer scale in the base system. The scale of 32 was chosen because it corresponds to the upper limit of reliable human color discrimination under normal viewing conditions — every integer step represents a perceptible difference. A value of 0 means not encoded: no conversion is performed for that axis, and it is treated as absent rather than as a default value.

Axis	Name	Range	Meaning	Analog
Color Axes — derivable from any digital color source				
H	Hue	RYGCBM wheel	R32, R16Y16, M32	Which color family
S	Shade	1–32	1 = black, 32 = white	Lightness
T	Tone	1–32	1 = gray, 32 = vivid	Saturation/chroma
Material Axes — require physical measurement				
L	Luster	1–32	1 = matte, 32 = glossy	Surface finish
F	Fluorescence	1–32	1 = none, 32 = vivid glow	UV emission

Color Axes — derivable from any digital color source

2.1 Hue

The Hue axis describes position on the RYGCBM color wheel using two components: one RGB primary and one adjacent YCM secondary. The two values always sum to 32. RGB primaries always appear in the first (primary) position; YCM secondaries always appear in the second

position. This ordering is enforced by the structure of valid pairs, not by sorting, and makes the notation unambiguous by definition.

Each RGB primary has exactly two valid YCM neighbors:

Primary (RGB)	Valid secondaries	Example notation	Color name	Interpretation
R	Y, M	R16Y16	Orange	Halfway R→Y
G	Y, C	G24Y8	Yellow-green	Mostly G, some Y
B	C, M	B8M24	Purple	Mostly M, some B

A pure hue (exactly on a named color) is written with a single component at 32: R32, Y32, G32, C32, B32, M32. The second component is omitted when its value is zero.

Achromatic colors (black, white, gray) carry no hue information. When Tone is 1 (fully desaturated), the Hue field is omitted entirely.

2.2 Shade

Shade encodes perceptual lightness from black (1) to white (32), corresponding to the L axis of OKLCh. OKLCh is perceptually uniform across the full gamut — equal steps in Shade correspond to equal perceived differences in lightness.

2.3 Tone

Tone encodes chroma (colorfulness) from fully desaturated (1) to the most vivid possible color (32), corresponding to the C axis of OKLCh. A Tone of 1 produces a pure neutral gray regardless of the Hue value; the Hue field should be omitted in this case.

Material Axes — require physical measurement

2.4 Luster

Luster encodes surface quality from matte (1) to glossy (32). It describes how light leaves the surface — scattered for matte, directed for gloss. Luster cannot be derived from digital color formats such as RGB or CMYK, which contain no surface information. It is measurable with a spectrophotometer and gloss meter and should always be populated when converting from physical color measurement. For paint, textiles, makeup, and nail polish, Luster is a required axis — these are materials, and describing their color without their surface finish is an incomplete description. Domain-specific standards may define named luster categories mapped onto the 1–32 scale; this is outside the scope of this specification.

When converting from any digital color format (RGB, CMYK, hex), Luster must be set to 0 (not encoded). When converting from physical color measurement, Luster should always be populated.

2.5 Fluorescence

Fluorescence encodes UV-activated light emission from 1 (none) to 32 (intensely fluorescent). It is relevant for paint, textiles, makeup, nail polish, security printing, and any application involving UV lighting. Like Luster, Fluorescence cannot be derived from digital color formats and must be

measured directly. When converting from digital sources, Fluorescence must be set to 0 (not encoded).

---

3. Notation Format

3.1 Full notation

A complete Hue_{32} color string has the form:

```
H:<hue> S:<shade> T:<tone> L:<luster> F:<fluorescence>
```

Examples:

```
H:R32      S:18  T:24  L:0  F:0    pure red, medium light, vivid
H:R16Y16   S:22  T:20  L:0  F:0    orange
H:R19M13   S:26  T:7   L:0  F:0    pastel pink
H:none     S:18  T:1   L:0  F:0    medium gray
H:B32      S:11  T:31  L:28 F:0    deep vivid blue, glossy
```

3.2 Hue string rules

- The RGB primary component always appears first.
- The YCM secondary component always appears second.
- A component with value 0 is omitted entirely (not written as X0).
- A pure hue is written as a single component: R32, not R32Y0.
- When Tone is 1 (fully desaturated), the Hue field is written as “none”.
- The two component values always sum to 32.

3.3 Zero and unspecified values

A value of 0 on any axis means not encoded. No conversion is performed for that axis. This is distinct from 16 (the explicit midpoint), which converts normally.

---

4. Precision Tiers

The base system uses integer steps on the 1–32 scale. Higher precision tiers subdivide each step using decimal notation, using the same format with decimal component values:

Tier	Step size	Steps	Intended use
Hue_{32}	1 (integer)	32	Paint, packaging, general use
Hue_{64}	0.5	64	Professional print, photography
Hue_{192}	0.25	192	Industrial, device drivers, spectrophotometry

The precision tier is implicit in the notation: any color string containing decimal values (e.g. R21.75Y10.25) is automatically a higher-precision encoding. Hue_{32} and Hue_{64} values are always a subset of Hue_{192} values, so precision tiers are fully backward compatible.

---

5. Reference Color Space

Hue_{32} uses OKLCh (the cylindrical form of OKLab) as its mathematical reference space. OKLCh is device-independent, based on human visual perception, and was designed specifically to address the perceptual non-uniformities present in CIE L*C*H* — particularly in the blue region, where equal steps in CIE chroma do not correspond to equal perceived differences. OKLCh is already used in CSS Color Level 4 and is supported by all major color science libraries.

5.1 Axis mappings

- Shade maps to L (lightness), scaled from [0, 1] to [1, 32].
- Tone maps to C (chroma), scaled from [0, 0.4] to [1, 32]. The ceiling of 0.4 provides headroom above the sRGB gamut for wide-gamut displays and physical pigments.
- Hue maps to the H angle (0–360°) via the six-segment wheel described in Section 6.

5.2 Out-of-gamut behavior

Because Hue_{32} is anchored to LCH rather than any device space, it can represent colors outside the gamut of any specific output device. Out-of-gamut handling is the responsibility of the output converter, not the notation system. A Hue_{32} value is always valid; whether a device can reproduce it is a separate question.

Implementations must flag out-of-gamut conversions to the caller but must not silently clip the Hue_{32} value itself.

---

6. The Hue Wheel

The RYGCBM wheel is divided into six segments. Each segment is bounded by one RGB primary and one adjacent YCM secondary. The OKLCh hue angles for each named color are derived from the sRGB primaries and secondaries converted through XYZ→OKLab→OKLCh:

RGB	YCM	Start angle	End angle	At t=0	At t=1
R	**Y**	29.2°	109.8°	t=0: all R	t=1: all Y
G	**Y**	109.8°	142.5°	t=0: all Y	t=1: all G
G	**C**	142.5°	194.8°	t=0: all G	t=1: all C
B	**C**	194.8°	264.1°	t=0: all C	t=1: all B
B	**M**	264.1°	328.4°	t=0: all B	t=1: all M
R	**M**	328.4°	29.2°*	t=0: all M	t=1: all R

* The R/M segment wraps through 0°. Angles between 328.4° and 360°, and between 0° and 29.2°, fall in this segment.

Within each segment, the 32 units are distributed proportionally to angular position. At the start boundary the named color at that boundary contributes all 32 units; at the end boundary the other named color contributes all 32 units. Values in between are linearly interpolated.

---

7. Minimal Reference Pseudocode

The following pseudocode describes a complete Hue_{32} implementation. Any language with a CIE LCH conversion library can implement this in approximately 35 lines of logic.

7.1 RGB → Hue_{32}

```
function rgb_to_hue32(r, g, b):
    L, C, H = rgb_to_oklch(r, g, b)        // use any OKLCh library
    shade  = 1 + (L / 100) * 31
    tone   = 1 + (C / 0.4) * 31
    if C < 2:                               // achromatic
        return { hue: none, shade, tone: 1, finish: 0 }
    rgb_name, rgb_val, ycm_name, ycm_val = angle_to_components(H)
    return { hue: (rgb_name, rgb_val, ycm_name, ycm_val), shade, tone, finish:
0 }
```

7.2 Angle to components

```
SEGMENTS = [                                // (rgb, ycm, start, end,
rgb_at_start)
    (R, Y,  40.0, 102.8, true ),
    (G, Y, 102.8, 136.0, false),            // t=0 is all-Y, t=1 is all-G
    (G, C, 136.0, 196.4, true ),
    (B, C, 196.4, 306.3, false),            // t=0 is all-C, t=1 is all-B
    (B, M, 306.3, 328.2, true ),
    (R, M, 328.2, 400.0, false),            // 400.0 = 40.0 + 360, wraparound
]

function angle_to_components(H):
    if H < 40.0: H = H + 360               // shift for wraparound segment
    for (rgb, ycm, start, end, rgb_at_start) in SEGMENTS:
        if start <= H <= end:
            t = (H - start) / (end - start)
            if rgb_at_start:
                rgb_val = (1 - t) * 32
                ycm_val = t * 32
            else:
                rgb_val = t * 32
                ycm_val = (1 - t) * 32
```

```
        return rgb, rgb_val, ycm, ycm_val
```

7.3 Hue_{32} → RGB

```
function hue32_to_rgb(hue32):
    L = (effective(hue32.shade) - 1) / 31 * 1.0
    C = (effective(hue32.tone)  - 1) / 31 * 0.4
    H = components_to_angle(hue32.hue)     // inverse of above
    return oklch_to_rgb(L, C, H)           // use any OKLCh library
                                           // clip to [0,255] and flag if
clipped

function effective(v):
    return 16 if v == 0 else v             // 0 means unspecified = midpoint
```

---

8. Conversion Support

Because Hue_{32} routes through OKLCh and CIE XYZ, any color system expressible in CIE XYZ is interoperable. CIE XYZ encompasses all practical color systems in current use. The conversion path is:

```
Source format → CIE XYZ → CIE LCH → Hue32
Hue32 → CIE LCH → CIE XYZ → Target format
```

Formats supported by the reference implementation:

- sRGB (0–255 per channel)
- Hex color strings (#RRGGBB)
- CMYK (0.0–1.0 per channel, neutral profile)
- CIE Lab and LCH directly

Formats reachable via standard library support:

- Adobe RGB, ProPhoto RGB, DCI-P3, and other RGB working spaces
- Munsell (via CIE Lab lookup tables)
- Spectral reflectance data (via CIE observer functions)

Formats requiring proprietary lookup tables (not in reference implementation):

- Pantone (Lab mappings exist commercially)
- RAL (Lab mappings are publicly available)
- NCS (Lab mappings are publicly available)

---

9. Conformance Requirements

A conforming Hue_{32} implementation must:

- Represent all five appearance axes (H, S, T, L, F) in its data structure. Color axes (H, S, T) are populated from digital sources. Material axes (L, F) default to 0 (not encoded) for digital sources and are populated from physical measurement.
- Treat a value of 0 on any axis as not encoded — perform no conversion for that axis. Do not substitute 16 for 0.
- Always place the RGB primary in the first hue position and the YCM secondary in the second position.
- Omit hue components with value 0 from the notation string.
- Write pure hues as a single component at 32 (R32, not R32Y0).
- Omit the Hue field and set Tone to 1 for achromatic colors.
- Flag out-of-gamut conversions to the caller without modifying the stored Hue_{32} value.
- Support at minimum the Hue_{32} (integer) precision tier. Hue_{64} and Hue_{192} are optional.
- Apply the snap rule: any computed component value below 0.25 must be set to 0 (not encoded). Values must not be rounded to the minimum valid value.
- Achieve round-trip accuracy within one step per tier: Hue_{32} dE < 0.0323, Hue_{64} dE < 0.0161, Hue_{192} dE < 0.0081, expressed as OKLab Euclidean distance.

---

10. Reference Implementation

A reference implementation in Python is provided alongside this specification as hue32.py. It uses the colour-science library (pip install colour-science) for CIE XYZ/OKLab/OKLCh conversions and supports RGB, hex, and CMYK input and output, using the five-axis notation H, S, T, L, F.

The reference implementation is the authoritative source for resolving ambiguities in this specification. When this document and the code disagree, the code is correct and this document should be updated.

---

11. The Scaling System

The three precision tiers of Hue_2 are not independent systems — they are a single system with a single 1–32 scale, subdivided by integer multiples. Understanding this relationship is essential for implementers who need to choose a precision tier, convert between tiers, or design systems that accept any tier as input.

11.1 Mathematical relationship between tiers

The three tiers are defined by how finely the 1–32 scale is subdivided:

```
Hue32   step = 1       positions per axis = 32
Hue64   step = 1/2     positions per axis = 64   = 32 × 2
Hue192  step = 1/4     positions per axis = 192  = 32 × 6
```

Every Hue_{32} value is a valid Hue_{64} value (since integers are a subset of half-integers), and every Hue_{64} value is a valid Hue_{192} value. The sets nest cleanly:

$$Hue_{32} \subset Hue_{64} \subset Hue_{192}$$

This means a Hue_{32} implementation can read Hue_{64} and Hue_{192} values by rounding to the nearest integer step, and a Hue_{192} implementation can output any value in any lower tier simply by rounding on output. No separate codepath is needed per tier — tier selection is a rounding decision, not a format change.

11.2 The tick ring encodes all three tiers

The Hue_{32} logo mark illustrates this directly. The 192-tick ring surrounding the color wheel has three visual weights:

Long marks (6 total) at the boundaries between named hue segments: R, Y, G, C, B, M. These correspond to the 6 segment boundaries of the hue wheel, or equivalently, every 32nd position on the Hue_{192} scale.

Medium marks (24 total, excluding the 6 long marks) at every 8th Hue_{192} position. These correspond to the 32 integer steps of the Hue_{32} scale — the positions a human eye can reliably distinguish.

Short marks (162 total) at every remaining Hue_{192} position. These are the quarter-step subdivisions available only to instruments.

The ring is therefore a physical representation of the precision hierarchy: read the long marks and you see the hue wheel; read the medium marks and you see Hue_{32}; read all marks and you see Hue_{192}.

11.3 Why 32 as the base

The choice of 32 as the base scale is grounded in human color discrimination. Research in color science suggests that under good viewing conditions, a trained observer can distinguish roughly 150–200 hue steps around the full wheel, about 25–30 lightness steps, and about 20–30 chroma steps depending on the hue region. A scale of 32 steps per axis sits comfortably within these limits. Because OKLCh is perceptually uniform, these steps are consistent across the wheel — a step near blue is equivalent to a step near yellow-green, which was not true of the previously used CIE L*C*H* space.

The Hue_{64} tier (64 steps) reaches the approximate limit of expert human discrimination under controlled conditions. The Hue_{192} tier (192 steps) exceeds it, and is intended for instrument-to-instrument communication, ICC profile precision, and archival color data where sub-perceptual precision is needed for downstream calculations.

11.4 Theoretical extension

The scaling pattern can extend further by the same rule — Hue_{34} (step = 1/8, 256 positions) or Hue_{34} (step = 1/16, 512 positions) — but no practical application for these tiers is known. High-end spectrophotometers typically report L*C*H* values to two decimal places, which corresponds to a precision well within Hue_{192}. Any extension beyond Hue_{192} would require a justification from instrumentation capability, not human perception.

Implementers should not define additional tiers without a corresponding real-world use case. The three defined tiers are expected to cover the full practical range from casual human use to the limits of current measurement technology.

12. Future Work and Open Questions